S03 UI Basics

CRE132 worksheet

What you'll make

A HUD (heads-up display) for S01's room: a **health bar** in the top left that shrinks as the player gets hurt, a **score** in the top right that counts S02's coins, and a **Restart** button at the top in the middle.

By the end you'll know how a **Canvas** and its **anchors** keep UI in place on any screen size, how scripts set **TextMeshPro** text and a **Filled** image, and how a button calls a public method with no extra code.

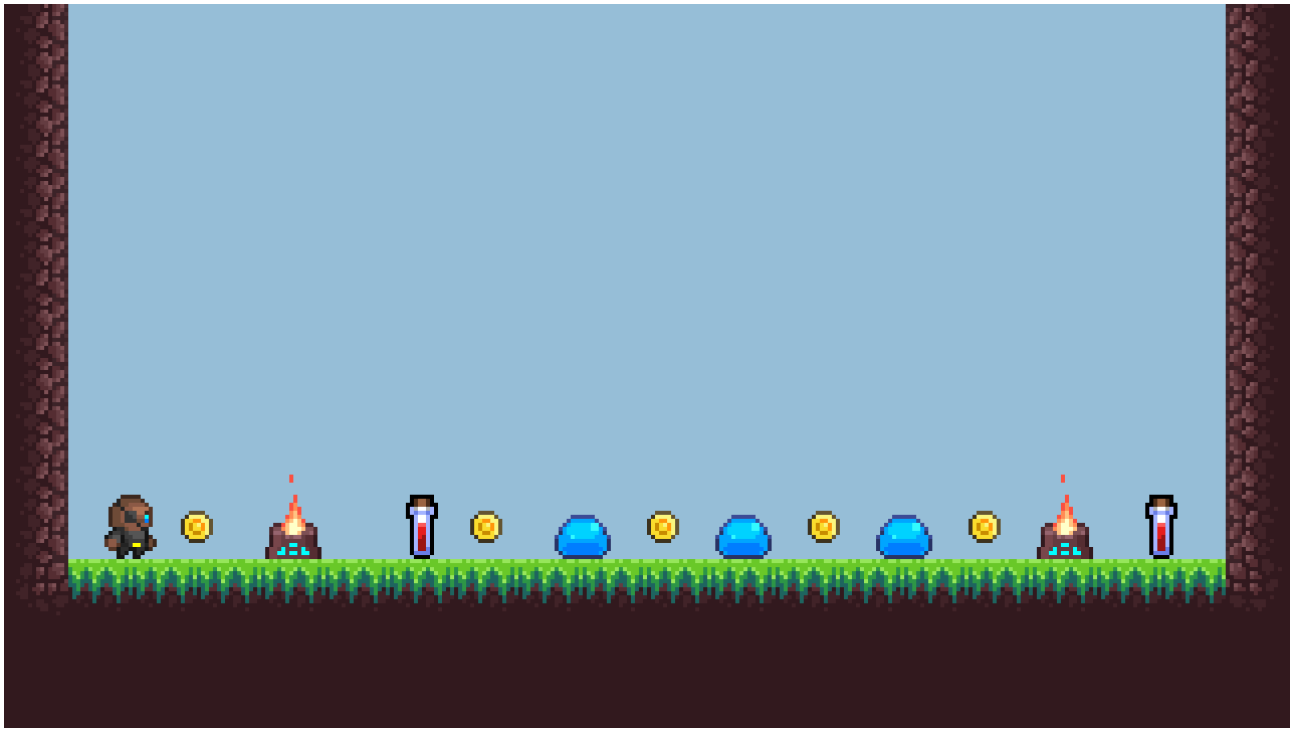


Figure 1: The finished scene: the player, coins, two fires, two potions and three training dummies.

Before you start

1. Unzip `CRE132_S03_UIBasics_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_S03_UIBasics_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Shared/S03_UIBasics`.
3. Press **Play**. Everything from S01 and S02 works: the fires hurt, the potions heal, J attacks, and coins add to the score in the Console. The HUD is there, but it never changes: it always says **5 / 5** and **Score: 0**, and the **Restart** button does nothing. Press Play again to stop.

The UI is already laid out under **HUD Canvas**: **Health Bar** (with a **Fill** and a **Health Text**), **Score Panel** (with **Score Text**) and **Restart Button**. You'll look at how it's built, then write the scripts that bring it to life.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. How the HUD is built

UI isn't made of sprites. It lives on a **Canvas**, and every UI object has a **Rect Transform** instead of a normal Transform. (To make your own later: **GameObject > UI > Canvas**, then **UI > Image**, **UI > Text - TextMeshPro** or **UI > Button - TextMeshPro**.)

1. Select **HUD Canvas**. The Canvas is huge in the Scene view (one unit per pixel), so press **F** to frame it. Its **Render Mode** is **Screen Space - Overlay**: it's drawn on top of everything, whatever the camera does.
2. Look at its **Canvas Scaler**: **UI Scale Mode** is **Scale With Screen Size**, **Reference Resolution** 1920 x 1080, **Match** 0.5. You lay the UI out for a 1080p screen and it scales up or down for others.
3. Select **Health Bar** and click the square **anchor presets** picture in its Rect Transform. It's anchored **top left**; the **Score Panel** is anchored top right and the **Restart Button** top centre. Anchors decide which part of the screen each element sticks to. (Hold **Shift+Alt** while picking a preset to set the pivot and position too.)
4. Press Play and change the Game view's size from **Free Aspect** to **16:9**, then **4:3**. Everything stays in its corner.
5. Select **Fill** (inside Health Bar). Its **Image Type** is **Filled** and **Fill Method** is **Horizontal**. Drag **Fill Amount** between 0 and 1 to see the bar empty and fill. That number is what the script will set. Put it back to 1.
6. Select **EventSystem**. It was made with the first UI object, and its **Input System UI Input Module** makes clicks work with the Input System.

2. The health bar: HealthBar

Make a folder `Assets/_game/Scripts/UI` and create a script in it called `HealthBar`:

```
// Fills an Image to match a Health component: full bar at full health, empty at none.
// The fill Image must have its Image Type set to Filled.
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class HealthBar : MonoBehaviour
{
    [SerializeField] private Health health;
    [SerializeField] private Image fillImage;
    [SerializeField] private TMP_Text healthText;

    private int shownHealth = -1;

    private void Update()
    {
        // When the player dies its Health is destroyed, so count that as zero.
        int currentHealth = 0;
        if (health != null)
        {
            currentHealth = health.CurrentHealth;
        }

        if (currentHealth == shownHealth)
        {
            return;
        }
        shownHealth = currentHealth;
```

```
        // fillAmount goes from 0 to 1. The cast to float stops whole-number division.
        if (health != null)
        {
            fillImage.fillAmount = (float)currentHealth / health.MaxHealth;
            healthText.text = currentHealth + " / " + health.MaxHealth;
        }
        else
        {
            fillImage.fillAmount = 0f;
            healthText.text = "0";
        }
    }
}
```

What's going on:

- **`using TMPro;`** and **`using UnityEngine.UI;`** give you `TMP_Text` (TextMeshPro text) and `Image`.
- **`fillAmount`** is the Fill Amount you dragged in step 1. Health divided by max health gives a number from 0 to 1; `(float)` stops whole-number division, as in S01's HealthTint.
- **`shownHealth`** remembers what's on screen, so the text is only rebuilt when health changes. It starts at −1 so the first frame always fills it in.
- **The `null` check:** when the player dies, S01's Health destroys it, so the bar counts that as 0 instead of throwing an error.
- **The HUD only reads** the player's health. It never changes it.

Add it to **Health Bar**, then fill its three slots: drag the **Player** into **Health**, **Fill** into **Fill Image** and **Health Text** into **Health Text**.

Test it: press Play and stand in a fire. The bar shrinks a fifth at a time and the text counts down. Drink a potion and it grows again.

3. The score: ScoreDisplay

Create `ScoreDisplay` in the `UI` folder:

```
// Shows the GameManager's score in a TextMeshPro text.
// It checks the score every frame but only changes the text when the score has
// changed.
using TMPro;
using UnityEngine;

public class ScoreDisplay : MonoBehaviour
{
    [SerializeField] private TMP_Text scoreText;

    // -1 so the text is filled in on the first frame, whatever the score is.
    private int shownScore = -1;

    private void Update()
    {
        if (GameManager.Instance == null)
        {
            return;
        }

        int score = GameManager.Instance.Score;
        if (score != shownScore)
        {
```

```
            shownScore = score;
            scoreText.text = "Score: " + score;
        }
    }
}
```

- **`GameManager.Instance.Score`** is S02's singleton, so the score display needs no slot for the GameManager.
- **The same "only when it changes" trick** as the health bar. Checking every frame is fine for a couple of numbers; S09 swaps it for events, so the UI only updates when something happens.

Add it to **Score Panel** and drag **Score Text** into the **Score Text** slot.

Test it: collect coins and watch the score go up in the top right.

4. The Restart button: SceneButtons

Make a folder `Assets/_game/Scripts/S03_UIBasics` and create `SceneButtons` in it:

```
// Public methods for UI buttons to call. In a Button's On Click () list, drag in the
// object with this script and pick the method from the dropdown.
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneButtons : MonoBehaviour
{
    public void RestartScene()
    {
        string sceneName = SceneManager.GetActiveScene().name;
        SceneManager.LoadScene(sceneName);
    }
}
```

- **A public method with no parameters** can be picked from a Button's **On Click ()** list. The button itself needs no code.
- **`RestartScene`** reloads the current scene, the same way S02's R key did.

Add it to **HUD Canvas**. Then select **Restart Button** and find **On Click ()** at the bottom of its **Button** component. The list is empty: click **+** to add an entry, drag **HUD Canvas** into its object slot (**None**), then pick **SceneButtons > RestartScene** from the dropdown next to it.

Test it: get hurt, collect some coins and click **Restart**. Health and coins reset; the score doesn't, because S02's GameManager survives the reload.

Check your work

- ☐ The health bar and its text go down in the fire and up with a potion. At zero the bar is empty.
- ☐ The score goes up with each coin.
- ☐ Restart reloads the level and keeps the score.
- ☐ At 16:9 and 4:3, the bar stays top left, the score top right and the button top centre.
- ☐ Compare with `CRE132_S03_UIBasics_Finished` if something behaves differently.

Key points

- **UI lives on a Canvas and uses Rect Transforms.** Anchors decide what each element sticks to when the screen changes size, and the Canvas Scaler scales the whole layout.
- **A public method with no parameters** (or one simple one) can be called from a Button without any extra code.

- **HUD scripts read game state** (Health, GameManager); they never change it.

If something goes wrong

- **NullReferenceException in HealthBar or ScoreDisplay:** a slot is empty. Check **Fill Image** and **Health Text** on Health Bar, and **Score Text** on Score Panel.
- **The bar never changes, but the text does:** the **Fill** image's **Image Type** isn't **Filled**.
- **The bar stays full:** the **Health** slot points at something other than the Player.
- **The button does nothing:** the On Click () entry still says **None**, or the dropdown says **No Function**. Drag in HUD Canvas and pick **SceneButtons > RestartScene** again.
- **"The type or namespace name 'TMP_Text' could not be found":** the `using TMPro;` line is missing.

Try it

1. Move the score to the bottom centre using only the anchor presets.
2. Add a **Coins Collected** text under the score. (Hint: duplicate **Score Text** with Ctrl+D.)
3. Change the bar's **Fill Origin** to **Right**, or the **Fill Method** to **Radial 360** for a circular timer.
4. Make a world-space health bar above a dummy: a new Canvas with **Render Mode** set to **World Space**, with its own `HealthBar` pointing at the dummy's Health.